

Capítulo 7 Aprendizaje por ensamble y Random Forests

La idea de fondo es simple: si le preguntas a mucha gente medio-experta y promedias sus respuestas, sueles acertar más que preguntándole a un solo experto, en ML lo mismo; juntar varios modelos “imperfectos, insuficientes” casi siempre da mejor resultado que confiar en uno solo. A ese grupo se le llama ensamble.

La forma más básica: votar. Entrenas varios modelos distintos (regresión logística, SVM, Random Forest, k-NN...) con los mismos datos y te quedas con lo que diga la mayoría. Suena demasiado simple, pero funciona, sin embargo, como una moneda desbalanceada al 51%: si la lanzas miles de veces, ese pequeño sesgo termina notándose. Para que funcione los modelos tienen que equivocarse en cosas distintas; si todos cometen el mismo error, votar no ayuda en nada, por eso conviene mezclar algoritmos diferentes entre sí. Una variante mejor es promediar probabilidades en vez de solo contar votos, porque así los votos "más seguros" pesan más.

Otra forma de generar diversidad es repartir los datos. También, en vez de usar distintos algoritmos, se usa el mismo algoritmo varias veces, pero cada vez con una porción distinta de los datos (con reemplazo = bagging, sin reemplazo = pasting).

Cada modelo individual queda un poco más "corto de vista", pero juntarlos reduce muchísimo el ruido/varianza del resultado final. Bonus: como cada uno entrena por separado, se puede paralelizar sin problema. Y como con el reemplazo cada modelo se queda sin ver como un tercio de los datos, ese sobrante sirve gratis para evaluarlo sin gastar un conjunto de validación aparte.

Random Forest es justo eso: muchos árboles entrenados con bagging, pero además, en cada punto de decisión del árbol, solo se le deja elegir entre un puñado de variables al azar (no todas). Esa aleatoriedad extra hace que los árboles se parezcan menos entre sí, y el conjunto termina generalizando mejor. Un plus muy práctico: te dice automáticamente qué variables importaron más. En el ejemplo del libro (flores Iris), el tamaño del pétalo resultó mucho más decisivo que el del sépalo.

- Boosting es la otra filosofía: en vez de entrenar todo en paralelo, entrena uno detrás de otro, y cada nuevo modelo se enfoca en corregir lo que el anterior hizo mal.
- AdaBoost le sube el "peso" a los casos que se fallaron para que el siguiente les preste más atención.
- Gradient Boosting va más directo: cada modelo nuevo aprende a predecir justo el error que dejó el anterior, y al final se suman todas las correcciones.
- XGBoost es la versión súper afinada y popular de esta idea (la que gana competencias).

Y el más sofisticado: stacking. En vez de combinar con una regla fija (votar, promediar), se entrena otro modelo cuyo único trabajo es aprender la mejor forma de mezclar las predicciones de los demás.

Conclusión: combinar modelos imperfectos pero diferentes casi siempre reduce el error, en paralelo (bagging) ataca la varianza, en secuencia (boosting) ataca el sesgo. Por eso Random

Forest y Gradient Boosting siguen siendo, hoy, de lo más usado en la industria para datos en tablas (rinden muy bien sin pedir tanto ajuste fino como una red neuronal).

Capítulo 8 **Reducción de Dimensionalidad**

Cuando un conjunto de datos tiene miles o incluso millones de variables por instancia el entrenamiento no solo se vuelve lento en encontrar una buena solución, además se vuelve difícil, a ese problema se le conoce como la maldición de la dimensionalidad. El origen del problema es geométrico y hasta contraintuitivo.

Nuestra intuición espacial funciona bien en dos o tres dimensiones pero deja de ser confiable a medida que aumentan los ejes. En un cuadrado un punto aleatorio casi nunca está cerca del borde, pero en un hipercubo de 10,000 dimensiones ocurre justo lo contrario: prácticamente todos los puntos terminan próximos a algún borde y algo similar sucede con las distancias: dos puntos aleatorios en un cuadrado están, en promedio, bastante cerca; en un espacio de un millón de dimensiones, esa misma distancia promedio se dispara. El resultado práctico es que los datos de entrenamiento quedan enormemente dispersos, por lo que cualquier instancia nueva tiende a estar lejos de todo lo que el modelo observó antes. Esto se traduce en predicciones menos confiables y en un riesgo mayor de sobreajuste, porque el modelo termina extrapolando en vez de interpolar. Aumentar la cantidad de datos para compensar esta dispersión no es una alternativa realista, el número de instancias necesarias crece exponencialmente con la dimensionalidad, al punto de que, con apenas cien variables, se requerirían más ejemplos que átomos existen en el universo.

Existen dos estrategias principales para enfrentar el problema:

- la primera es la proyección, en la mayoría de los casos reales las variables no están dispersas de forma uniforme sino que muchas son casi constantes o están fuertemente correlacionadas entre sí, de modo que los datos ocupan en realidad un subespacio de dimensión mucho menor que la original. Proyectar los datos sobre ese subespacio permite reducir su dimensionalidad sin perder gran parte de su estructura. Sin embargo, este enfoque falla cuando el subespacio no es plano sino curvo; el caso clásico es el Swiss roll, una superficie enrollada sobre sí misma en tres dimensiones ya que la proyección terminaría aplastando capas que en realidad están separadas. Para esos casos se recurre al manifold learning que en lugar de proyectar intenta modelar directamente la forma curva sobre la que se distribuyen los datos, dicha estrategia parte de la llamada hipótesis de la variedad, según la cual la mayoría de los conjuntos de datos de alta dimensión se concentran cerca de variedades de dimensión mucho menor. Se advierte que trabajar en ese espacio reducido no siempre simplifica la tarea posterior de clasificación o regresión; en ocasiones ocurre lo contrario.
- El PCA (Análisis de Componentes Principales) sigue siendo la técnica más utilizada. Su lógica consiste en identificar el eje que conserva la mayor varianza posible al proyectar los datos sobre él, luego un segundo eje ortogonal al primero que capture la varianza restante, y así sucesivamente. El cálculo se realiza mediante descomposición en valores singulares.

Un indicador especialmente útil es la razón de varianza explicada, que permite decidir cuántas componentes conservar en función de qué proporción de la información original se desea preservar, por ejemplo: el 95% en lugar de fijar ese número de forma arbitraria. Aplicado a MNIST, este criterio reduce las 784 variables originales a poco más de 150, con una pérdida de información mínima. Existen además variantes orientadas a distintos escenarios prácticos: una versión aproximada y más veloz cuando se buscan pocas componentes, y otra incremental, pensada para conjuntos de datos que no caben completos en memoria.

El Kernel PCA extiende esta misma lógica al terreno no lineal, aplicando el mismo principio del kernel utilizado en las máquinas de vectores de soporte, lo que resulta especialmente útil para "desenrollar" estructuras como el Swiss roll, algo que el PCA convencional no logra. El LLE (Locally Linear Embedding) aborda el problema desde otro ángulo: en lugar de proyectar, analiza cómo se relaciona cada instancia con sus vecinos más cercanos y construye una representación de menor dimensión que preserve esas relaciones locales, aunque a costa de perder fidelidad en las distancias globales.

Conclusion, la maldición de la dimensionalidad ofrece una justificación matemática no solo intuitiva de por qué reducir el número de variables puede mejorar la capacidad de generalización de un modelo.

PCA se ha convertido en una herramienta transversal dentro del flujo de trabajo de ciencia de datos: como paso de preprocesamiento como recurso de visualización en dos o tres dimensiones y como mecanismo de compresión. Conviene recordar, que toda reducción de dimensionalidad implica cierta pérdida de información, por lo que se recomienda entrenar primero con los datos originales y recurrir a estas técnicas únicamente **cuando el entrenamiento resulte demasiado lento o exista sospecha razonable de ruido o redundancia.**

